

# Optimal Control for Linear Dynamical Systems and Quadratic Cost ("LQR")

Reinforcement Learning

---

D. Sorokin

April 27, 2026

# Bellman's Curse of Dimensionality

1.  $n$ -dimensional state space
2. Number of states grows exponentially in  $n$  (for fixed number of discretizations levels per coordinate)
3. In practice
  - 3.1 Discretizations is considered only computationally feasible up to 5 or 6 dimensional state spaces even when using
    - 3.1.1 Variable resolution discretizations
    - 3.1.2 Highly optimized implementations
  - 3.2 Function approximation might or might not work in practice

## Part I: Linear Quadratic Regulator (LQR)

- ✓ Exact solution for continuous state-space optimal control via linear algebra —  $O(H \times n^3)$
- ✓ Closed-loop stability, controllability and observability
- ✓ Extensions: affine systems, stochastic noise, nonlinear systems (linearization), penalizing control changes, LTV systems, trajectory following
- ✓ Iterative LQR (iLQR) for general nonlinear systems

## Part II: Kalman Filter

- ✓ Optimal state estimator for linear systems with Gaussian noise —  $O(H \times n^3)$
- ✓ Predict–update algorithm; steady-state Riccati equation
- ✓ Duality with LQR; LQG control and the separation principle
- ✓ Extended Kalman Filter (EKF) for nonlinear systems

# Linear Quadratic Regulator (LQR)

Linear dynamical system:

$$x_{t+1} = Ax_t + Bu_t$$

$x_t$ : state at time  $t$

$u_t$ : action at time  $t$

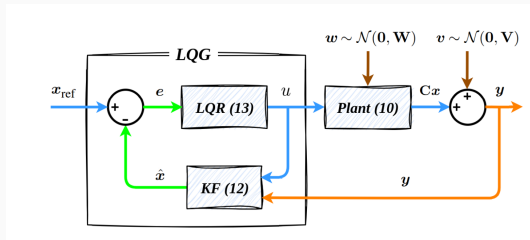
Quadratic cost function (i.e -reward):

$$g(x_t, u_t) = x_t^T Q x_t + u_t^T R u_t$$

$Q, R$  - symmetric, positive definite

$(Q \succ 0, R \succ 0)$

$Q \succ 0 \iff \forall z : z^T Q z > 0$



## Reminder: symmetric matrix

symmetric matrix is rotation - rescaling - rotation back:

$$Q = V\Lambda V^T$$

$V$  - orthogonal i.e.  $V^T V = I$

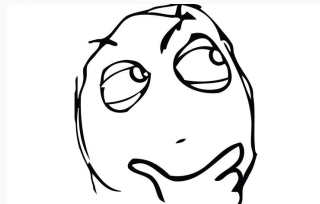
$\Lambda$  - diagonal

## What if $Q$ is not symmetric?

$$g(x_t, u_t) = \frac{1}{2}x_t^T Q x_t + \frac{1}{2}u_t^T R u_t$$
$$x^T Q x = x^T \frac{Q+Q^T}{2} x + x^T \frac{Q-Q^T}{2} x = x^T \frac{Q+Q^T}{2} x$$

since  $\frac{x^T Q x}{2} - \frac{x^T Q^T x}{2} = \frac{x^T Q x}{2} - \frac{(x^T Q x)^T}{2} = 0$

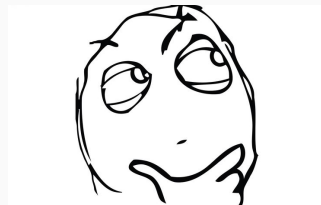
only symmetric part  $\frac{Q+Q^T}{2}$  matters!



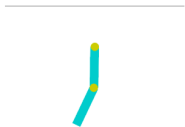
## What if $Q$ is not positive definite?

$$g(x_t, u_t) = x_t^T Q x_t + u_t^T R u_t$$

if  $z^T Q z < 0$  you can infinitely minimize cost function  $g$   
so the task does not have a finite optimal solution.



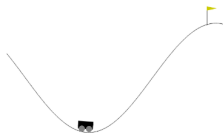
## Classic Control



Acrobot



Cart Pole



Mountain Car  
Continuous



Mountain Car



Pendulum



Back-up step for  $i+1$  steps to go:

$$J_{i+1}(s) = \min_u g(s, u) + \sum_{s'} P(s'|s, u) J_i(s')$$

LQR:

$$J_{i+1}(x) = \min_u x^T Q x + u^T R u + J_i(x' = Ax + Bu)$$

## LQR value iteration: $J_1$

Cost for zero steps-to-go:  $J_0(x) = x^T P_0 x$

Cost for one step-to-go:

$$J_1(x) = \min_u \left[ x^T Q x + u^T R u + (Ax + Bu)^T P_0 (Ax + Bu) \right]$$

$$\nabla_u [J_1(x)] = 2Ru + 2B^T P_0 (Ax + Bu) = 0$$

$$\rightarrow u = -(R + B^T P_0 B)^{-1} B^T P_0 A x = K_1 x$$

$$J_1(x) = x^T P_1 x$$

$$P_1 = Q + K_1^T R K_1 + (A + B K_1)^T P_0 (A + B K_1)$$

$$K_1 = -(R + B^T P_0 B)^{-1} B^T P_0 A$$

Cost for N steps will have the same form (!):

$$J_N(x) = x^T P_N x$$

## Value iteration solution to LQR

Set  $P_0 = 0$

for  $i = 1, 2, 3, \dots$

$$K_i = -(R + B^T P_{i-1} B)^{-1} B^T P_{i-1} A$$

$$P_i = Q + K_i^T R K_i + (A + B K_i)^T P_{i-1} (A + B K_i)$$

The optimal policy for a i-step horizon is given by:  $\pi(x) = K_i x$

The cost-to-go function for a i-step horizon is given by:  $J_i(x) = x^T P_i x$

- Fact: Guaranteed to converge to the infinite horizon optimal policy if and only if the dynamics  $(A, B)$  is such that there exists a policy that can drive the state to zero.
- Often most convenient to use the steady-state  $K$  for all times.

# Why Use Steady-State $K$ for All Times?

## 1. $K_i$ converges as the horizon grows

$P_0, P_1, P_2, \dots$  is monotonically increasing and bounded above by the cost of any fixed stabilizing policy  $\Rightarrow$  converges to  $P_\infty$ , so  $K_i \rightarrow K_\infty$ .

For most of a long trajectory the gain has already converged — only the last few steps near the terminal time differ.

## 2. Infinite-horizon problem has a constant gain by definition

No terminal time  $\Rightarrow$  number of steps remaining never changes  $\Rightarrow$  stationary Bellman equation  $\Rightarrow K_\infty$  is exactly optimal at every step.

## 3. Practical advantages

- Solve the algebraic Riccati equation **once** offline
- Runtime: single matrix multiply  $u_t = K_\infty x_t$ , no time index needed
- $K_\infty$  is **guaranteed to stabilize** the closed loop; intermediate  $K_i$  are not

$$K = -(R + B^T P B)^{-1} B^T P A$$

$$P = Q + K^T R K + (A + B K)^T P (A + B K) =$$

$$Q + A^T P A + A^T P B K + K^T (R + B^T P B) K + K^T B^T P A =$$

$$P = Q + A^T P A + A^T P B K \quad (\text{Discrete time Riccati equation})$$

## Infinite-horizon, continuous time

$$\dot{x} = Ax + Bu$$

$$J(x) = \int_0^\infty (x^T Qx + u^T Ru) dt = x_0^T Px_0 + \int_0^\infty \left( \frac{d}{dt}(x^T Px) + x^T Qx + u^T Ru \right) dt =$$

$$x_0^T Px_0 + \int_0^\infty (\dot{x}^T Px + x^T P\dot{x} + x^T Qx + u^T Ru) dt =$$

$$x_0^T Px_0 + \int_0^\infty (Ax + Bu)^T Px + x^T P(Ax + Bu) + x^T Qx + u^T Ru) dt =$$

$$x_0^T Px_0 + \int_0^\infty (x^T A^T Px + u^T B^T Px + x^T PAx + x^T PBu + x^T Qx + u^T Ru) dt =$$

$$J(x) = x_0^T Px_0 +$$

$$\int_0^\infty (x^T (A^T P + PA + Q - PBR^{-1}B^T P)x + (u + R^{-1}B^T Px)^T R(u + R^{-1}B^T Px)) dt$$

## Infinite-horizon, continuous time

$$J(x) = x_0^T P x_0 + \int_0^\infty (x^T (A^T P + PA + Q - PBR^{-1}B^T P)x) dt + \int_0^\infty (u + R^{-1}B^T P x)^T R (u + R^{-1}B^T P x) dt$$

Choose matrix  $P$  and  $K$  such that:

$$u = Kx$$

$$K = -R^{-1}B^T P$$

$$A^T P + PA + Q - PBR^{-1}B^T P = 0 \quad (\text{Riccati equation})$$

$$J(x) = x_0^T P x_0$$

$$x_{t+1} = Ax_t + Bu_t$$

$$g(x_t, u_t) = x_t^T Q x_t + u_t^T R u_t$$

= for keeping a linear system at the all-zeros state while preferring to keep the control input small.

- Extensions make it more generally applicable:
  - Affine systems
  - Systems with stochasticity
  - Regulation around non-zero fixed point for non-linear systems
  - Penalization for change in control inputs
  - Linear time varying (LTV) systems
  - Trajectory following for non-linear systems



$$x_{t+1} = Ax_t + Bu_t + c$$

$$g(x_t, u_t) = x_t^T Q x_t + u_t^T R u_t$$

- Optimal control policy remains linear, optimal cost-to-go function remains quadratic
- Two avenues to do derivation:
  - 1. Re-derive the update, which is very similar to what we did for standard setting
  - 2. Re-define the state as:  $z_t = [x_t; 1]$ , then we have:

$$z_{t+1} = \begin{bmatrix} x_{t+1} \\ 1 \end{bmatrix} = \begin{bmatrix} A & c \\ 0 & 1 \end{bmatrix} \times \begin{bmatrix} x_t \\ 1 \end{bmatrix} + \begin{bmatrix} B \\ 0 \end{bmatrix} u_t = A' z_t + B' u_t$$

## LQR Ext1: stochastic system

$$x_{t+1} = Ax_t + Bu_t + w_t$$

$$g(x_t, u_t) = x_t^T Q x_t + u_t^T R u_t$$

$w_t, t=0,1,\dots$  are zero mean and independent

- Exercise: work through similar derivation as we did for the deterministic case, but which will now have expectations.
- Result:
  - Same optimal control policy
  - Cost-to-go function is almost identical: has one additional term which depends on the variance in the noise (and which cannot be influenced by the choice of control inputs)

## LQR Ext2: non-linear systems

**Nonlinear system:**  $x_{t+1} = f(x_t, u_t)$

**We can keep the system at the state  $x^*$  iff**

$$\exists u^* \text{ s.t. } x^* = f(x^*, u^*)$$

**Linearizing the dynamics around  $x^*$  gives:**

$$x_{t+1} \approx f(x^*, u^*) + \frac{\partial f}{\partial x}(x^*, u^*)(x_t - x^*) + \frac{\partial f}{\partial u}(x^*, u^*)(u_t - u^*)$$

**Equivalently:**

$$x_{t+1} - x^* \approx A(x_t - x^*) + B(u_t - u^*)$$

**Let  $z_t = x_t - x^*$ , let  $v_t = u_t - u^*$ , then:**

$$z_{t+1} = Az_t + Bv_t, \text{ cost} = z_t^T Qz_t + v_t^T Rv_t$$

$$v_t = Kz_t \Rightarrow u_t - u^* = K(x_t - x^*) \Rightarrow u_t = u^* + K(x_t - x^*)$$

## LQR Ext3: Penalize for Change in Control Inputs

- Standard LQR:

$$x_{t+1} = Ax_t + Bu_t$$

$$g(x_t, u_t) = x_t^T Q x_t + u_t^T R u_t$$

- When run in this format on real systems: often high frequency control inputs get generated. Typically highly undesirable and results in poor control performance.
- Why?
- Solution: frequency shaping of the cost function. Can be done by augmenting the system with a filter and then the filter output can be used in the quadratic cost function. (See, e.g., Anderson and Moore.)
- Simple special case which works well in practice: penalize for change in control inputs. – How ??

## LQR Ext3: Penalize for Change in Control Inputs

- Standard LQR:

$$x_{t+1} = Ax_t + Bu_t$$

$$g(x_t, u_t) = x_t^T Q x_t + u_t^T R u_t$$

- How to incorporate the change in controls into the cost/reward function?
  - Soln. method A: explicitly incorporate into the state by augmenting the state with the past control input vector, and the difference between the last two control input vectors.
  - Soln. method B: change of control input variables.

## LQR Ext3: Penalize for Change in Control Inputs

- Standard LQR:

$$x_{t+1} = Ax_t + Bu_t$$

$$g(x_t, u_t) = x_t^T Q x_t + u_t^T R u_t$$

- Introducing change in controls  $\Delta u$ :

$$\begin{bmatrix} x_{t+1} \\ u_t \end{bmatrix} = \begin{bmatrix} A & B \\ 0 & I \end{bmatrix} \begin{bmatrix} x_t \\ u_{t-1} \end{bmatrix} + \begin{bmatrix} B \\ I \end{bmatrix} \Delta u_t$$

$$\text{cost} = x'^T Q' x' + \Delta u^T R' \Delta u$$

$$Q' = \begin{bmatrix} Q & 0 \\ 0 & R \end{bmatrix},$$

$R'$  = penalty for change in controls

$$x_{t+1} = A_t x_t + B_t u_t$$

$$g(x_t, u_t) = x_t^T Q x_t + u_t^T R u_t$$

## LQR Ext4: Linear Time Varying (LTV) Systems

Set  $P_0 = 0$

for  $i = 1, 2, 3, \dots$

$$K_i = -(R_{H-i} + B_{H-i}^T P_{i-1} B_{H-i})^{-1} B_{H-i}^T P_{i-1} A_{H-i}$$

$$P_i = Q_{H-i} + K_i^T R_{H-i} K_i + (A_{H-i} + B_{H-i} K_i)^T P_{i-1} (A_{H-i} + B_{H-i} K_i)$$

The optimal policy for a  $i$ -step horizon is given by:  $\pi(x) = K_i x$

The cost-to-go function for a  $i$ -step horizon is given by:  $J_i(x) = x^T P_i x$



## LQR Ext5: Trajectory Following for Non-Linear Systems

- A state sequence  $x_0^*, x_1^*, \dots, x_H^*$  is a feasible target trajectory if and only if

$$\exists u_0^*, u_1^*, \dots, u_{H-1}^* : \forall t \in \{0, 1, \dots, H-1\} : x_{t+1}^* = f(x_t^*, u_t^*)$$

- Problem statement:

$$\min_{u_0, u_1, \dots, u_{H-1}} \sum_{t=0}^{H-1} (x_t - x_t^*)^T Q (x_t - x_t^*) + (u_t - u_t^*)^T R (u_t - u_t^*)$$

$$\text{s.t. } x_{t+1} = f(x_t, u_t)$$

- Transform into linear time varying case (LTV):

$$x_{t+1} \approx f(x_t^*, u_t^*) + \frac{\partial f}{\partial x}(x_t^*, u_t^*)(x_t - x_t^*) + \frac{\partial f}{\partial u}(x_t^*, u_t^*)(u_t - u_t^*)$$

$$x_{t+1} - x_{t+1}^* \approx A_t(x_t - x_t^*) + B_t(u_t - u_t^*)$$

## LQR Ext5: Trajectory Following for Non-Linear Systems

- Transform into linear time varying case (LTV):

$$x_{t+1} \approx f(x_t^*, u_t^*) + \frac{\partial f}{\partial x}(x_t^*, u_t^*)(x_t - x_t^*) + \frac{\partial f}{\partial u}(x_t^*, u_t^*)(u_t - u_t^*)$$

$$x_{t+1} - x_{t+1}^* \approx A_t(x_t - x_t^*) + B_t(u_t - u_t^*)$$

- Now we can run the standard LQR back-up iterations.
- Resulting policy at  $i$  time-steps from the end:

$$u_{H-i} - u_{H-i}^* = K_i(x_{H-i} - x_{H-i}^*)$$

- The target trajectory need not be feasible to apply this technique, however, if it is infeasible then there will be an offset term in the dynamics:

$$x_{t+1} - x_{t+1}^* \approx f(x_t^*, u_t^*) - x_{t+1}^* + A_t(x_t - x_t^*) + B_t(u_t - u_t^*)$$

- How about this general optimal control problem?

$$\min_{u_0, \dots, u_H} \sum_{t=0}^H g(x_t, u_t)$$

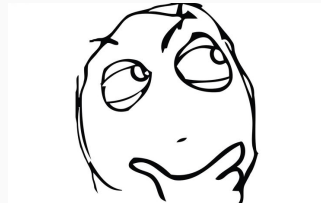
## Iteratively Apply LQR

Initialize the algorithm by picking either (a) A control policy  $\pi^{(0)}$  or (b) A sequence of states  $x_0^{(0)}, x_1^{(0)}, \dots, x_H^{(0)}$  and control inputs  $u_0^{(0)}, u_1^{(0)}, \dots, u_H^{(0)}$ . With initialization (a), start in Step (1). With initialization (b), start in Step (2). Iterate the following:

1. Execute the current policy  $\pi^{(i)}$  and record the resulting state-input trajectory  $x_0^{(i)}, u_0^{(i)}, x_1^{(i)}, u_1^{(i)}, \dots, x_H^{(i)}, u_H^{(i)}$ .
2. Compute the LQ approximation of the optimal control problem around the obtained state-input trajectory by computing a first-order Taylor expansion of the dynamics model, and a second-order Taylor expansion of the cost function.
3. Use the LQR back-ups to solve for the optimal control policy  $\pi^{(i+1)}$  for the LQ approximation obtained in Step (2).
4. Set  $i = i + 1$  and go to Step (1).

# Wait...

Can we actually observe  $x$ ?



## Part II: Kalman Filter

- So far: assume full state observation  $x_t$  is known
- In practice: only a noisy, partial observation  $y_t$  is available
- ✓ Kalman Filter: optimal state estimator for linear systems with Gaussian noise
  - Exact closed-form solution (no discretization)
  - Running time  $O(H \times n^3)$  — same as LQR
  - Dual problem to LQR

## Kalman Filter: Setup

Linear dynamical system with process and observation noise:

$$x_{t+1} = Ax_t + Bu_t + w_t, \quad w_t \sim \mathcal{N}(0, W)$$

$$y_t = Cx_t + v_t, \quad v_t \sim \mathcal{N}(0, V)$$

$x_t \in \mathbb{R}^n$ : state     $y_t \in \mathbb{R}^m$ : observation

$w_t$ : process noise     $v_t$ : measurement noise

$W \succeq 0$ : process noise covariance     $V \succ 0$ : measurement noise covariance

**Goal:** given observations  $y_0, y_1, \dots, y_t$  and inputs  $u_0, \dots, u_{t-1}$ , compute the posterior:

$$p(x_t \mid y_0, \dots, y_t)$$

**Key fact:** this posterior is Gaussian at all times — it suffices to track its mean  $\hat{x}_t$  and covariance  $P_t$ .

# Kalman Filter: Algorithm

Maintain Gaussian belief  $p(x_t|y_{0:t}) = \mathcal{N}(\hat{x}_{t|t}, P_{t|t})$ .

**Prediction step** (propagate through dynamics):

$$\hat{x}_{t|t-1} = A\hat{x}_{t-1|t-1} + Bu_t \quad (\text{predicted mean})$$

$$P_{t|t-1} = AP_{t-1|t-1}A^T + W \quad (\text{predicted covariance})$$

$P_{t|t-1}$ : covariance of the prediction error  $x_t - \hat{x}_{t|t-1}$

$AP_{t-1|t-1}A^T$ : posterior uncertainty propagated through dynamics

$W$ : new uncertainty added by process noise — irreducible even with perfect prior knowledge

**Update step** (incorporate new observation  $y_t$ ):

$$K_t = P_{t|t-1}C^T \left( CP_{t|t-1}C^T + V \right)^{-1} \quad (\text{Kalman gain})$$

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t(y_t - C\hat{x}_{t|t-1})$$



## KF Update: Deriving the Posterior Covariance

Posterior estimate:  $\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t(y_t - C\hat{x}_{t|t-1})$  for some gain  $K_t$ .

**Posterior error**  $e_{t|t} = x_t - \hat{x}_{t|t}$ :

$$\begin{aligned}e_{t|t} &= x_t - \hat{x}_{t|t-1} - K_t(\underbrace{Cx_t + v_t}_{y_t} - C\hat{x}_{t|t-1}) \\&= e_{t|t-1} - K_t(Ce_{t|t-1} + v_t) \\&= (I - K_tC) e_{t|t-1} - K_tv_t\end{aligned}$$

**Posterior covariance**  $P_{t|t} = \mathbb{E}[e_{t|t}e_{t|t}^T]$

(cross terms vanish:  $v_t$  independent of  $e_{t|t-1}$ , both zero-mean):

$$\begin{aligned}P_{t|t} &= (I - K_tC) \underbrace{\mathbb{E}[e_{t|t-1}e_{t|t-1}^T]}_{P_{t|t-1}} (I - K_tC)^T + K_t \underbrace{\mathbb{E}[v_tv_t^T]}_V K_t^T \\&= (I - K_tC) P_{t|t-1} (I - K_tC)^T + K_tVK_t^T\end{aligned}$$

## KF Update: Deriving the Optimal Kalman Gain

Expand  $P_{t|t} = (I - K_t C) P_{t|t-1} (I - K_t C)^T + K_t V K_t^T$ :

$$P_{t|t} = P_{t|t-1} - K_t C P_{t|t-1} - P_{t|t-1} C^T K_t^T + K_t \underbrace{(C P_{t|t-1} C^T + V)}_{S_t} K_t^T$$

Minimize  $\text{tr}(P_{t|t})$  over  $K_t$ , set derivative to zero:

$$\frac{\partial \text{tr}(P_{t|t})}{\partial K_t} = -2P_{t|t-1} C^T + 2K_t S_t = 0$$

$$\boxed{K_t = P_{t|t-1} C^T (C P_{t|t-1} C^T + V)^{-1}}$$

Substituting back, the last two terms cancel:

$$P_{t|t} = P_{t|t-1} - K_t C P_{t|t-1} = (I - K_t C) P_{t|t-1}$$

$P_{t|t} \preceq P_{t|t-1}$  always — every observation reduces uncertainty.

## Kalman Filter: Covariance Riccati Equation

The prior covariance  $\Sigma_t = P_{t|t-1}$  satisfies a Riccati recursion.

Substituting  $P_{t|t}$  into the prediction step:

$$\Sigma_{t+1} = W + A\Sigma_t A^T - A\Sigma_t C^T \left( C\Sigma_t C^T + V \right)^{-1} C \Sigma_t A^T$$

**Recall: LQR discrete-time ARE (for comparison):**

$$P = Q + A^T P A - A^T P B \left( R + B^T P B \right)^{-1} B^T P A$$

- Identical structure:  $A \leftrightarrow A^T$ ,  $B \leftrightarrow C^T$ ,  $Q \leftrightarrow W$ ,  $R \leftrightarrow V$
- Converges to a steady-state covariance  $\Sigma$  as  $t \rightarrow \infty$

**Steady-state Kalman gain:**

$$K = \Sigma C^T \left( C\Sigma C^T + V \right)^{-1}$$

## Duality: LQR and Kalman Filter

The two problems are exact duals — replace matrices as follows:

| LQR (control)                                   | Kalman Filter (estimation)                 |
|-------------------------------------------------|--------------------------------------------|
| $A$                                             | $A^T$                                      |
| $B$                                             | $C^T$                                      |
| $Q$ (state cost)                                | $W$ (process noise)                        |
| $R$ (input cost)                                | $V$ (measurement noise)                    |
| $P_i$ (cost-to-go matrix)                       | $\Sigma_t$ (error covariance)              |
| $K_i = -(R + B^T P_{i-1} B)^{-1} B^T P_{i-1} A$ | $K_t = \Sigma C^T (C \Sigma C^T + V)^{-1}$ |

- LQR minimizes cost going *backward* in time
- Kalman Filter minimizes estimation error going *forward* in time
- Same Riccati structure, same  $O(n^3)$  per step

# LQG Control: Combining LQR and Kalman Filter

**Setting:** linear system, quadratic cost, Gaussian noise (LQG).

**Problem:** we observe  $y_t$  but not  $x_t$  — LQR requires the state.

## Separation Principle:

1. Design the LQR controller as if  $x_t$  were fully observed  $\Rightarrow$  gain  $K$
2. Design the Kalman Filter to estimate  $\hat{x}_{t|t}$  from  $y_0, \dots, y_t$
3. Apply  $u_t = K\hat{x}_{t|t}$

This is **optimal**: the LQG optimal policy has exactly this structure.

$$\hat{x}_{t+1|t} = A\hat{x}_{t|t} + Bu_t$$

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_{KF}(y_t - C\hat{x}_{t|t-1})$$

$$u_t = K_{LQR}\hat{x}_{t|t}$$

## Extended Kalman Filter (EKF)

**Setting:** nonlinear dynamics and observation model:

$$x_{t+1} = f(x_t, u_t) + w_t, \quad w_t \sim \mathcal{N}(0, W)$$

$$y_t = h(x_t) + v_t, \quad v_t \sim \mathcal{N}(0, V)$$

**Idea:** linearize around the current estimate  $\hat{x}_{t|t-1}$ , then run standard KF.

**Prediction step:**

$$\hat{x}_{t|t-1} = f(\hat{x}_{t-1|t-1}, u_t) \quad (\text{use nonlinear } f)$$

$$A_t = \frac{\partial f}{\partial x}(\hat{x}_{t-1|t-1}, u_t), \quad P_{t|t-1} = A_t P_{t-1|t-1} A_t^T + W$$

**Update step:**

$$C_t = \frac{\partial h}{\partial x}(\hat{x}_{t|t-1}), \quad K_t = P_{t|t-1} C_t^T \left( C_t P_{t|t-1} C_t^T + V \right)^{-1}$$

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t (y_t - h(\hat{x}_{t|t-1}))$$

## Where Does LQR/LQG Fit in RL?

| Setting                            | Method         | Quality                |
|------------------------------------|----------------|------------------------|
| Known linear model, no noise       | LQR            | globally optimal       |
| Known linear model, Gaussian noise | LQG            | globally optimal       |
| Known nonlinear model              | iLQR / DDP     | locally optimal        |
| Unknown model, learn online        | model-based RL | asymptotically optimal |
| No model at all                    | model-free RL  | general, sample-hungry |

LQR/LQG plays three roles in modern RL:

1. **Fast subroutine:** model-based RL methods (e.g. GPS) run iLQR at each iteration after fitting a local linear model from data
2. **Baseline:** near-linear systems are well served by LQR; a model-free agent that cannot match it has not learned much
3. **Warm start:** LQR policy provides a strong initial policy for policy search in nonlinear settings